Abstract

This project focuses on detecting fake news using machine learning techniques. The system uses a Logistic Regression model trained on a dataset of real and fake news articles. Text data is preprocessed and converted into numerical form using TF-IDF vectorization. A Streamlit-based dashboard is used to display predictions and analysis. The system classifies news as fake or real but does not support real-time news, making it a prototype model for educational purposes. The rapid growth of digital media and online news platforms has significantly increased the spread of misinformation and fake news. This project presents an AI-based Fake News Detection System that aims to automatically classify news articles as fake or real using machine learning techniques. The system is built using the Python programming language and utilizes libraries such as Pandas, Scikit-learn, and Streamlit for data processing, model training, and user interface development. Although the system demonstrates effective classification on the given dataset, it is limited by the use of static and relatively small data, and it does not support real-time news analysis. Therefore, this project serves as a prototype that highlights the application of machine learning in fake news detection and provides a foundation for future enhancements using larger datasets and advanced models.

List of Figures & Tables:

- Figure 1:System Architecture Diagram
- Figure 2:Confusion Matrix
- Figure 3:ROC Curve
- Figure 4:Fake vs Real Pie Chart
- Table 1:Dataset Description
- Table 2:Model Performance Metrics

List of Abbreviations:

- ML–Machine Learning
- TF-IDF–Term Frequency Inverse Document Frequency
- UI– User Interface
- ROC– Receiver Operating Characteristic
- AUC–Area Under Curve

Index

Sr. No.	Title	Page No.
1.	Introduction	1-2
2.	Literature Review	3
3.	System Analysis	4
4.	System Design	5
5.	Implementation	6-13
6.	Testing and Results	14-17
7.	Advantages & Limitations	18
8.	Conclusion	19
9.	Future Scope	20
10.	References	21

Introduction

• Background:

In recent years, the rapid growth of the internet and social media platforms has transformed the way people consume news. Information is now easily accessible and spreads quickly across digital platforms. However, this has also led to the widespread circulation of fake news, which can mislead people, create confusion, and influence public opinion. Detecting fake news manually is difficult, time-consuming, and often unreliable. Therefore, there is a need for automated systems that can assist in identifying fake news efficiently using advanced technologies like Machine Learning.

• Problem Statement:

The main problem addressed in this project is the automatic identification of fake news from textual content. With the increasing volume of online news, it becomes challenging to verify the authenticity of every article manually. This project aims to develop a system that can analyze news text and classify it as fake or real using machine learning techniques. However, the system is limited to a predefined dataset and does not handle real-time or live news data.

• Objectives:

The primary objectives of this project are:

- To develop a machine learning model for fake news classification To preprocess and clean textual news data effectively
- To convert text into numerical form using TF-IDF To build an interactive dashboard using Streamlit
- To evaluate model performance using metrics like accuracy, confusion matrix, and ROC curve.

• Scope of the Project:

This project is designed as a prototype system for fake news detection. It works on a static dataset of labeled news articles and provides predictions based on trained data. The system allows users to analyze random news and upload CSV files for batch processing. However, it does not support real-time news analysis, multilingual data, or large-scale deployment. The scope is limited to demonstrating the application of machine learning in text classification.

• Methodology Overview:

The methodology of the project follows a structured approach:

- **Data Collection**–Load fake and real news datasets
- **Data Preprocessing**–Clean and normalize text data
- **Feature Extraction**–Convert text into vectors using TF-IDF
- **Model Training** – Train Logistic Regression model
- **Prediction** – Classify news as fake or real
- **Visualization**–Display results using charts and graphs in Streamlit

Literature Review

Fake news detection has become an important topic because of the increasing use of the internet and social media. Many researchers have worked on different methods to identify whether news is real or fake. Earlier, fake news was checked manually by experts or using simple rule-based methods. These methods were slow, time-consuming, and not suitable for handling large amounts of data. To solve this problem, machine learning techniques were introduced. Machine learning models such as **Naive Bayes, Support Vector Machine (SVM), and Logistic Regression** are commonly used for fake news detection. These models analyze the text of news articles and classify them into fake or real categories. Among these, Logistic Regression is simple, fast, and works well for binary classification problems.

To use machine learning, text data must be converted into numbers. Techniques like **Bag of Words (BoW)** and **TF-IDF** are used for this purpose. TF-IDF is widely used because it gives more importance to important words and reduces the effect of common words. Recently, advanced methods like **deep learning models (LSTM, BERT)** have been used. These models can understand the meaning and context of the text better, but they require large datasets and more computational power.

In this project, a simple and efficient approach is used with TF-IDF and Logistic Regression. This makes the system easy to implement and suitable for small datasets. However, it has some limitations, such as not understanding context and not working with real-time news.

System Analysis

• Existing System:

- Manual fact-checking
- Time-consuming
- Not scalable

• Proposed System:

- Automated fake news detection
- Uses machine learning
- Provides visual dashboard

• Requirement Specifications:

➢ **Functional Requirements:**

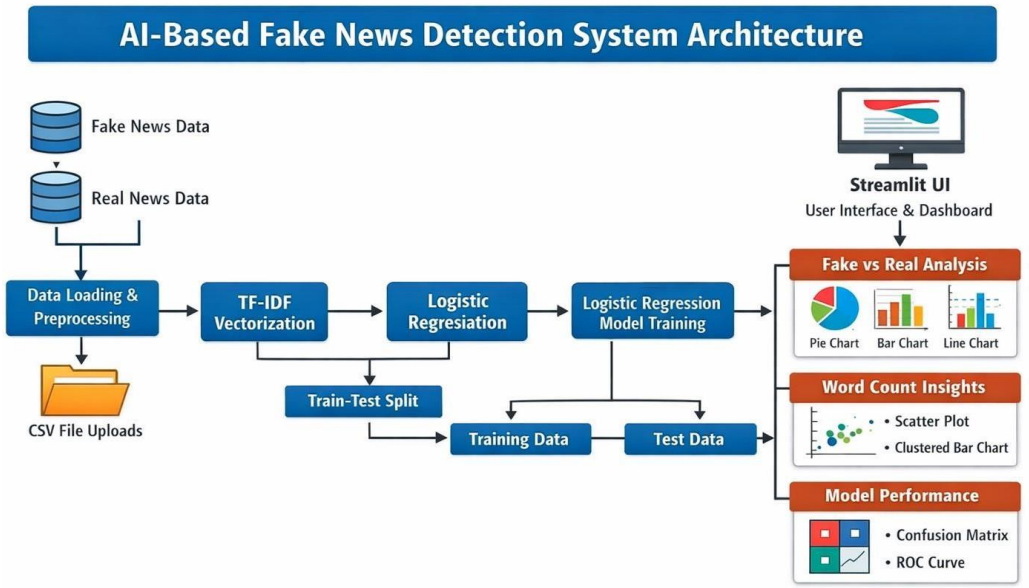
- Load dataset
- Clean text data
- Train model
- Predict fake/real news
- Upload .CSV files
- Display graphs and results

➢ **Non-Functional Requirements:**

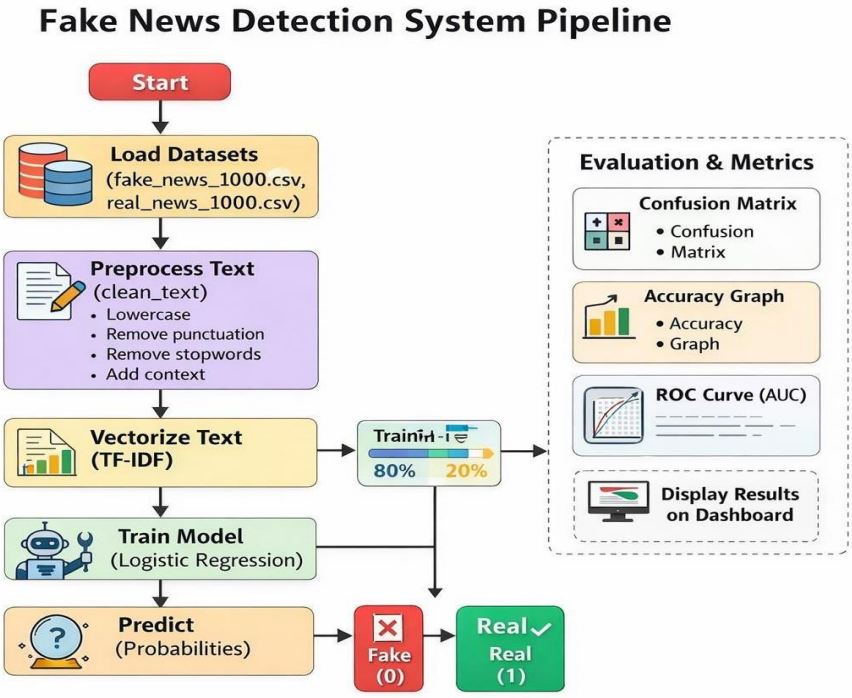
- Fast processing
- User-friendly interface
- Moderate accuracy
- Runs on standard systems

System Design

➢ System Architecture:



➢ System Pipeline:



Implementation

```
import streamlit as st
import pandas as pd
import re
import random
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, roc_curve, auc
from sklearn.model_selection import train_test_split

# ----------------------------
# LOAD DATA
# ----------------------------
fake = pd.read_csv("fake_news_1000.csv")
real = pd.read_csv("real_news_1000.csv")

data = pd.concat([fake, real], axis=0)
data = data.sample(frac=1).reset_index(drop=True)

# ----------------------------
# CLEAN TEXT
# ----------------------------
def clean_text(text):
    text = text.lower()
    text = re.sub(r'\W', ' ', text)
    text = re.sub(r'\s+', ' ', text)
    return text
```

```
data['text'] = data['text'].apply(clean_text)


# ----------------------------
# MODEL (TRAIN-TEST SPLIT)
# ----------------------------
X = data['text']
y = data['label']

vectorizer = TfidfVectorizer(stop_words='english', max_df=0.7, ngram_range=(1,2))

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Vectorize
X_vec_train = vectorizer.fit_transform(X_train)
X_vec_test = vectorizer.transform(X_test)

# Train model
model = LogisticRegression(max_iter=1000, class_weight='balanced')
model.fit(X_vec_train, y_train)

# ----------------------------
# STREAMLIT UI
# ----------------------------
st.title("□ AI-Based Fake News Detection Dashboard")
st.subheader("□ Automated News Analysis (No Manual Input)")
# ----------------------------
# SESSION STATE
# ----------------------------
```

```
if "combined_data" not in st.session_state:
    st.session_state.combined_data = pd.DataFrame()

if "filtered_data" not in st.session_state:
    st.session_state.filtered_data = data.copy()

# ----------------------------
# RANDOM NEWS
# ----------------------------
if st.button("Check Random News"):
    news = random.choice(list(data['text']))
    vect = vectorizer.transform([news])
    probs = model.predict_proba(vect)[0]
    pred = 1 if probs[1] > 0.45 else 0

    st.write("□ News:", news)
    st.write(f"□ Fake: {probs[0]*100:.2f}% | □ Real: {probs[1]*100:.2f}%")

    if pred == 1:
        st.success("✅Real News")
    else:
        st.error("❌Fake News")

    st.session_state.filtered_data = data[data['text'] == news]
# ----------------------------
# FILE UPLOAD
# ----------------------------
uploaded_files = st.file_uploader("Upload CSV Files", accept_multiple_files=True)
if uploaded_files:
```

```
  for file in uploaded_files:
        df = pd.read_csv(file)

 df['text'] = df['text'].apply(lambda x: clean_text("news report " + x))
        vect = vectorizer.transform(df['text'])
        probs = model.predict_proba(vect)
        preds = [1 if p[1] > 0.45 else 0 for p in probs]
        df['label'] = preds

        st.session_state.combined_data = pd.concat(
            [st.session_state.combined_data, df],
            ignore_index=True
        )

# ----------------------------
# RESET BUTTON
# ----------------------------
if st.button("Reset Uploaded Data"):
    st.session_state.combined_data = pd.DataFrame()
    st.session_state.filtered_data = data.copy()
    st.session_state.fake_count = 5
    st.session_state.real_count = 5
    st.success("Data cleared successfully!")
# ----------------------------
# SELECTION OF FAKE & REAL
# ----------------------------
if not st.session_state.combined_data.empty:
    st.write("□ Combined Uploaded Data")
    fake_data = st.session_state.combined_data[st.session_state.combined_data['label']==0]
    real_data = st.session_state.combined_data[st.session_state.combined_data['label']==1]
```

```
# ----- Input number of Fake News -----
fake_input = st.text_input(
    f"Enter number of Fake News (0-{len(fake_data)}):",
    value=str(min(5, len(fake_data))),
    key="fake_count_input"
)
try:
    fake_n = int(fake_input)
    if fake_n < 0: fake_n = 0
    elif fake_n > len(fake_data): fake_n = len(fake_data)
except:
    fake_n = min(5, len(fake_data))
selected_fake = fake_data.head(fake_n)
# ----- Input number of Real News -----
real_input = st.text_input(
    f"Enter number of Real News (0-{len(real_data)}):",
    value=str(min(5, len(real_data))),
    key="real_count_input"
)
try:
    real_n = int(real_input)
    if real_n < 0: real_n = 0
    elif real_n > len(real_data): real_n = len(real_data)
except:
    real_n = min(5, len(real_data))
selected_real = real_data.head(real_n)
st.session_state.filtered_data = pd.concat([selected_fake, selected_real], ignore_index=True)
st.write(f"□ Showing {fake_n} Fake and {real_n} Real News")
st.write(st.session_state.filtered_data)
```

```
# ----------------------------
# DASHBOARD (ALL ORIGINAL GRAPHS)
# ----------------------------
st.subheader("□ AI Analysis Dashboard")
filtered_data = st.session_state.filtered_data

# PIE CHART
counts = filtered_data['label'].value_counts()
labels = ["Fake" if i==0 else "Real" for i in counts.index]
colors = ["red" if i==0 else "green" for i in counts.index]

fig1, ax1 = plt.subplots()
ax1.pie(counts, labels=labels, autopct='%1.1f%%', colors=colors)
ax1.set_title("Fake vs Real Distribution")
st.pyplot(fig1)

# BAR CHART
fig_bar, ax_bar = plt.subplots()
ax_bar.bar(labels, counts.values, color=colors)
ax_bar.set_title("Fake vs Real Count")
st.pyplot(fig_bar)

# LINE CHART
trend = filtered_data['label'].rolling(5).mean()
st.line_chart(trend)

# WORD COUNT
filtered_data['word_count'] = filtered_data['text'].apply(lambda x: len(x.split()))
```

```
# SCATTER
fig2, ax2 = plt.subplots()
color_map = ['red' if i==0 else 'green' for i in filtered_data['label']]
ax2.scatter(filtered_data['word_count'], filtered_data['label'],
    c=color_map, alpha=0.6)
ax2.set_title("Word Count vs News Type")
st.pyplot(fig2)

# CLUSTERED BAR
bins = [0,10,20,30,50,100]
filtered_data['range'] = pd.cut(filtered_data['word_count'], bins)
grouped = filtered_data.groupby(['range','label']).size().unstack(fill_value=0)
grouped.columns = ["Fake" if col==0 else "Real" for col in grouped.columns]

fig3, ax3 = plt.subplots()
grouped.plot(kind='bar', ax=ax3, color=['red','green'][:len(grouped.columns)])
ax3.set_title("Clustered Bar Chart")
st.pyplot(fig3)

# ----------------------------
# MODEL PERFORMANCE (TRAIN-TEST DATA)
# ----------------------------
st.subheader("□ Model Performance (Test Data)")

# CONFUSION MATRIX
y_pred = model.predict(X_vec_test)
cm = confusion_matrix(y_test, y_pred)
fig_perf1, ax_perf1 = plt.subplots()
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm',
```

```
    xticklabels=['Fake','Real'],
     yticklabels=['Fake','Real'])
ax_perf1.set_title("Confusion Matrix (Test Data)")
st.pyplot(fig_perf1)

# ACCURACY GRAPH
if cm.size == 4:
    TN, FP, FN, TP = cm.ravel()
    fake_acc = TN/(TN+FP) if (TN+FP)!=0 else 0
    real_acc = TP/(TP+FN) if (TP+FN)!=0 else 0

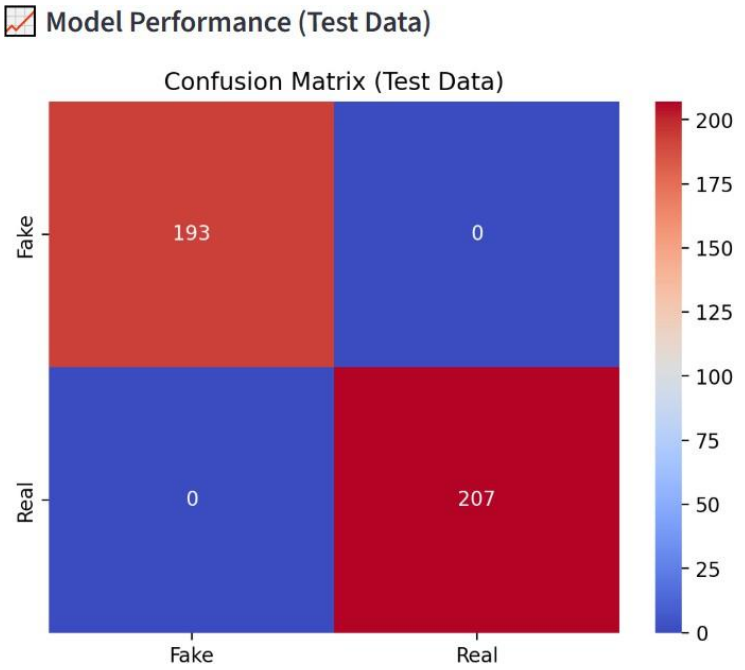
    fig_perf2, ax_perf2 = plt.subplots()
    ax_perf2.bar(['Fake','Real'], [fake_acc, real_acc], color=['red','green'])
    ax_perf2.set_title("Accuracy (Test Data)")
    st.pyplot(fig_perf2)

# ROC CURVE
y_prob = model.predict_proba(X_vec_test)[:,1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

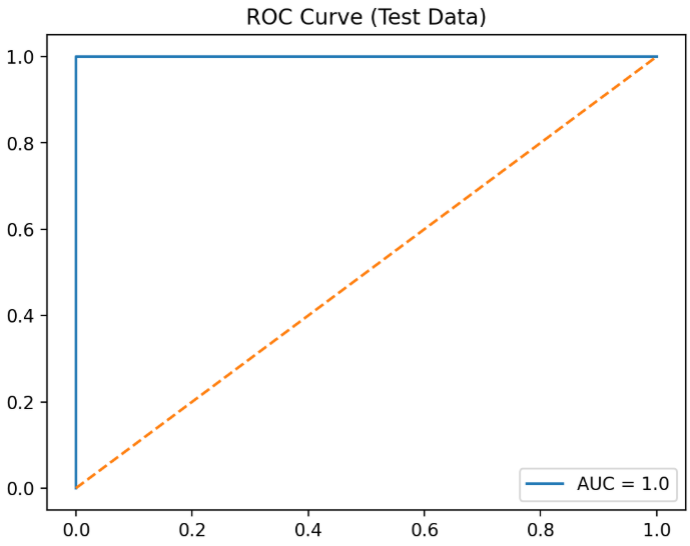
fig_perf3, ax_perf3 = plt.subplots()
ax_perf3.plot(fpr, tpr, label="AUC = " + str(round(roc_auc,2)))
ax_perf3.plot([0,1],[0,1],'--')
ax_perf3.set_title("ROC Curve (Test Data)")
ax_perf3.legend()
st.pyplot(fig_perf3)
```

Testing and Results

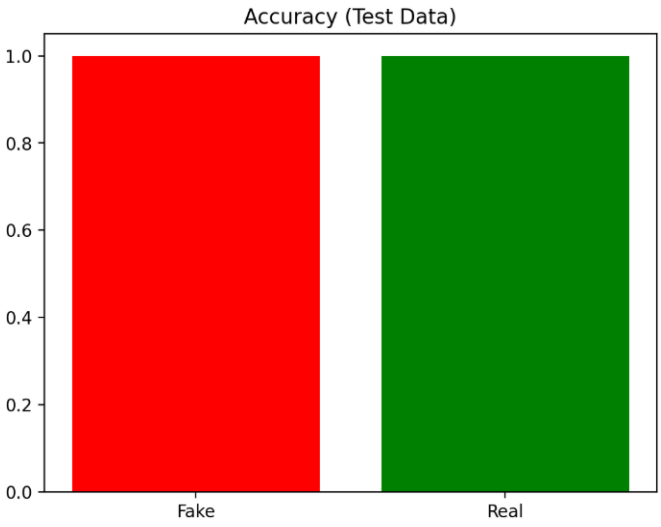
- Model tested using test dataset:-
- Confusion matrix used for evaluation



- ROC curve plotted

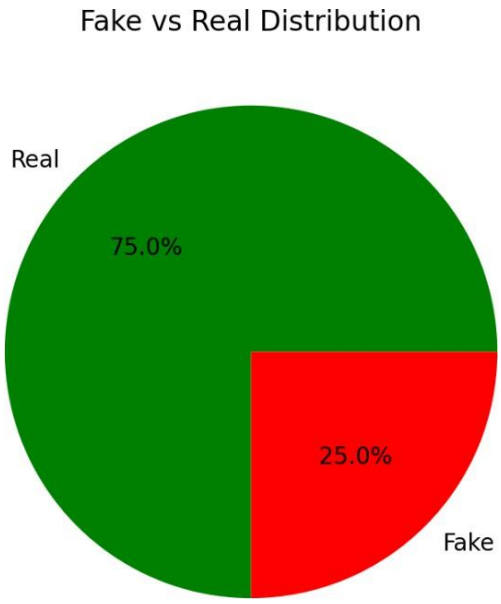


➢ Accuracy measured for fake and real news

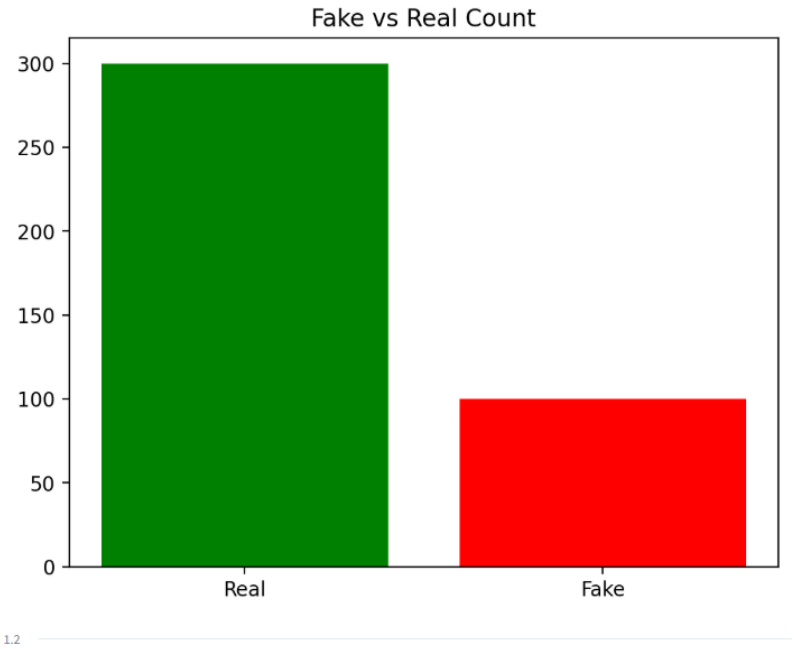


➢ Dashboard shows:

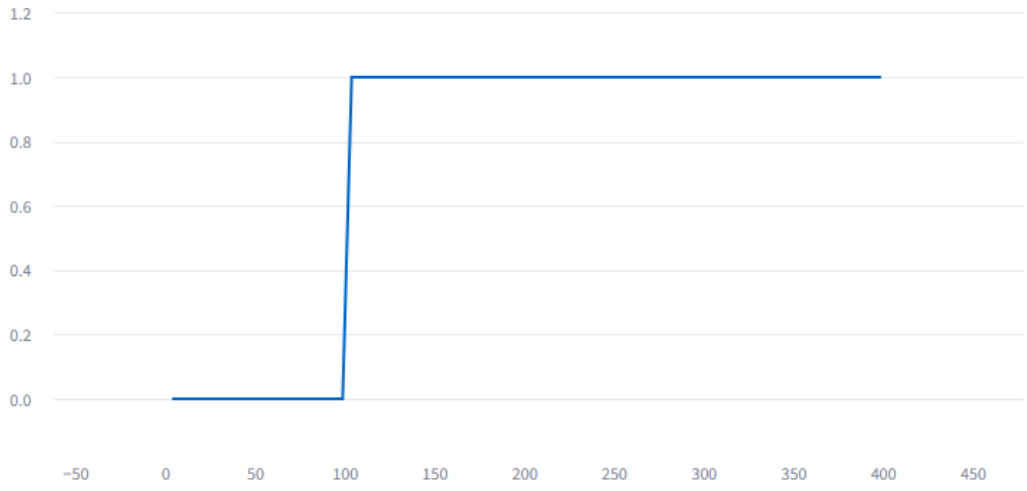
- **Pie chart**



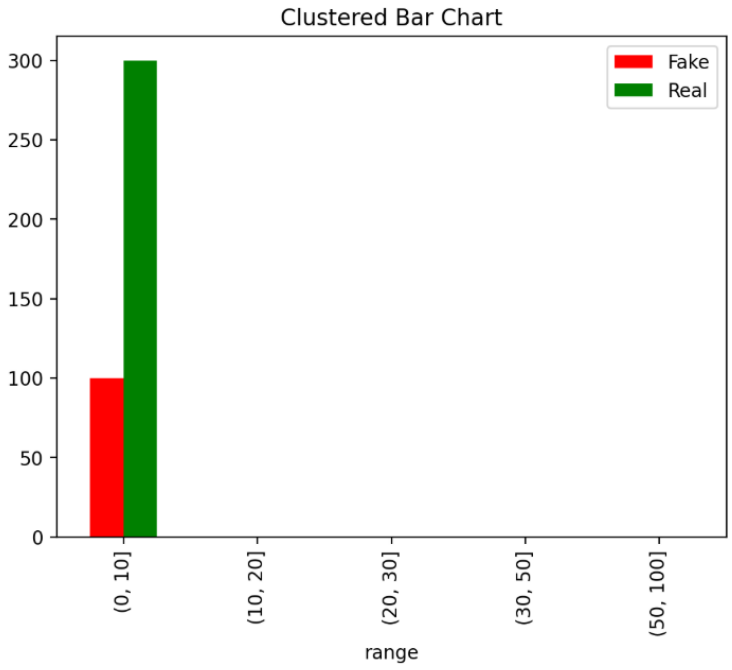
- **Bar chart**



- **Line chart**



• **Clustered Bar chart**



Advantages

- Automatic fake news detection
- Easy to use dashboard
- Graphical analysis
- Fast prediction
- Useful for social media and news platforms
- Reduces human effort in fact-checking
- Helps in preventing the spread of misinformation

Limitations

- No real-time news support
- Limited dataset

Conclusion

The project successfully detects fake news using machine learning. It provides good accuracy on trained data but is limited due to lack of real-time data and small dataset. This project successfully detects fake and real news using Machine Learning. The Streamlit dashboard makes the system interactive and easy to use. It helps users quickly identify fake news and analyze data visually.

Future Scope

- Use real-time news APIs
- Implement deep learning models(BERT, LSTM)
- Improve dataset size
- Improve accuracy
- Social Media Monitoring System
- Supports Big Dataset

References

➢ Research Papers:

- Alshuwaier, F. A., & Alsulaiman, F. A. (2025). Fake News Detection Using Machine Learning and Deep Learning Algorithms: A Comprehensive Review and Future Perspectives. Computers, 14(9), 394.
- Reddy, S. V., Wadekar, A., Ghorpade, B., Wagh, S., & Sampate, P. (2025). Fake News Detection Using Machine Learning and NLP. International Journal of Creative Research Thoughts (IJCRT).
- Tiwari, S., & Jain, S. (2024). Fake News Detection Using Machine Learning Algorithms. Proceedings of the International Conference on Computing Sciences (ICCS 2023).
- Puri, R. (2024). Fake News Detection: A Comprehensive Study on Analysis and Modern Classification Techniques. Research Gate Publication.
- Fake News Detection using Python and Machine Learning. (2024). Procedia Computer Science, Volume 233, Pages 763–771.
- Jouhar, J., et al. (2024). Fake news detection using Python and machine learning. Procedia Computer Science.
- Alghamdi, J., Lin, Y., & Luo, S. (2026). Machine learning and deep learning approaches for fake news detection in multilingual contexts: A systematic literature review. Multimedia Tools and Applications (Springer).